Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Certified Kubernetes Administrator (CKA) Training

TGS Ref No: TGS-2025054612

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	02 July 2026	First version — step-by-step guide to all 31 CKA labs across five domains (Cluster Architecture & Installation, Workloads & Scheduling, Services & Networking, Storage, Troubleshooting); MD and DOCX generated from one source	Tertiary Infotech Academy Pte Ltd

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

This Learner Guide accompanies the WSQ Certified Kubernetes Administrator (CKA) course (TGS-2025054612). It contains step-by-step lab instructions for all 31 hands-on labs covering the five CKA exam domains. Labs run in the browser on Killercoda — no local installation required.

CKA Exam Domains

Domain	Weight	Labs
Cluster Architecture, Installation & Configuration	25%	1–8
Workloads & Scheduling	15%	9–16
Services & Networking	20%	17–22
Storage	10%	23–25
Troubleshooting	30%	26–31

Before You Start — Setup & Prerequisites

0.1 What you need

Tool	Used for	Where to get it
kubectl	All labs — the Kubernetes CLI	kubernetes.io/docs/tasks/tools/
kubeadm	Labs 1–5: cluster bootstrap and upgrade	Pre-installed on Killercoda kubeadm scenario
Helm 3	Lab 6: Helm chart deployment	helm.sh/docs/intro/install/
etcdctl (v3)	Lab 5: etcd backup and restore	github.com/etcd-io/etcd/releases
crictl	Labs 1–5: container runtime debugging	github.com/kubernetes-sigs/cri-tools
Killercoda	All labs — free browser Kubernetes environment	killercoda.com
killer.sh	Day 4 mock-exam practice	killer.sh (CKA simulator)

0.2 Exam-speed aliases — set these before every session

The CKA exam is 2 hours, 100% hands-on. These aliases save many keystrokes and are expected in a timed environment.

```
alias k=kubectl
export do="--dry-run=client -o yaml"
source <(kubectl completion bash)
complete -o default -F __start_kubectl k
```

Note: On the real CKA exam these aliases are pre-configured. Practise using them in every lab so the muscle memory is there on exam day.

DAY 1 — DOMAIN 1: Cluster Architecture, Installation & Configuration (Labs 1–8)

Lab 1 — kubeadm Prerequisites and Container Runtime

Domain: Cluster Architecture, Installation & Configuration · Topic: Cluster installation · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Goal

Prepare two clean Ubuntu nodes for a Kubernetes cluster install: load kernel modules, configure sysctls, install containerd as the CRI, and install kubeadm/kubelet/kubectl from the official pkgs.k8s.io repository. CKA tests this sequence in troubleshooting questions where a node is NotReady due to a misconfigured runtime.

What you'll build

Configure kernel modules, sysctls, containerd (SystemdCgroup=true), and install pinned Kubernetes packages on both nodes.

Step 1 — Load kernel modules

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter
```

Step 2 — Set required sysctls

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system
```

Step 3 — Disable swap

```
sudo swapoff -a
sudo sed -i '/ swap / s/^(.*)$/#\1/g' /etc/fstab
```

Step 4 — Install containerd

```
sudo apt update && sudo apt install -y containerd
```

```
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
/etc/containerd/config.toml
sudo systemctl restart containerd && sudo systemctl enable containerd
```

Note: SystemdCgroup = true aligns containerd's cgroup driver with kubelet. A mismatch is the #1 cause of 'node NotReady' in fresh clusters.

Step 5 — Install kubeadm, kubelet, kubectl

```
sudo apt install -y apt-transport-https ca-certificates curl gpg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.31/deb/ /' | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt update && sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Step 6 — Verify

```
kubeadm version
kubectl version --client
sudo systemctl status containerd --no-pager | head
```

Test it: kubeadm version shows v1.31.x, containerd is active, and crictl reports the runtime version.

Lab 2 — kubeadm Bootstrap: init, kubeconfig, Worker Join, Cluster PKI

Domain: Cluster Architecture, Installation & Configuration · Topic: Cluster installation · Duration: 60 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Goal

Run kubeadm init to create the control plane, configure kubectl access via kubeconfig, join a worker node using the bootstrap token, explore the PKI certificates under /etc/kubernetes/pki/, take an etcd snapshot, and inspect static pod manifests. The CKA exam always includes an etcd backup/restore question.

What you'll build

Run kubeadm init, copy admin.conf, join worker, explore PKI, take etcd snapshot with etcdctl.

Step 1 — Initialize the control plane

```
kubeadm init \
  --kubernetes-version=v1.35.0 \
  --pod-network-cidr=192.168.0.0/16 \
  --service-cidr=10.96.0.0/12 \
  --apiserver-advertise-address=$(hostname -I | awk '{print $1}')
```

Step 2 — Configure kubectl

```
mkdir -p $HOME/.kube
cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes
```

Step 3 — Explore static pod manifests

```
ls /etc/kubernetes/manifests/
grep 'image:' /etc/kubernetes/manifests/kube-apiserver.yaml
```

Step 4 — Explore cluster PKI

```
ls /etc/kubernetes/pki/
kubeadm certs check-expiration
openssl x509 -in /etc/kubernetes/pki/ca.crt -noout -dates
```

Step 5 — Take an etcd snapshot

```
export ETCDCTL_API=3
export ETCDCTL_ENDPOINTS=https://127.0.0.1:2379
export ETCDCTL_CACERT=/etc/kubernetes/pki/etcd/ca.crt
export ETCDCTL_CERT=/etc/kubernetes/pki/etcd/server.crt
export ETCDCTL_KEY=/etc/kubernetes/pki/etcd/server.key
etcdctl snapshot save /opt/etcd-backup-$(date +%Y%m%d).db
etcdctl snapshot status /opt/etcd-backup-$(date +%Y%m%d).db --write-
out=table
```

Note: Always set ETCDCTL_API=3. etcd backup/restore is guaranteed to appear on the CKA exam.

Step 6 — Join worker node

```
# On control plane -- generate join command
kubeadm token create --print-join-command
# Run the output on WORKER NODE
# Back on control plane:
kubectl get nodes -o wide
```

Test it: kubectl get nodes shows both nodes (NotReady until CNI installed). etcd snapshot file exists at /opt/.

Lab 3 — Install Calico CNI and Verify Cross-Node Pod Ping

Domain: Cluster Architecture, Installation & Configuration · Topic: CNI networking · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Goal

Install Calico as the CNI plugin using the Tigera Operator pattern, watch nodes transition to Ready, deploy test pods on different nodes, and validate cross-node pod-to-pod communication. The pod CIDR in Calico must match the --pod-network-cidr from kubeadm init.

What you'll build

Apply tigera-operator.yaml + custom-resources.yaml, watch nodes go Ready, ping across nodes.

Step 1 — Confirm nodes are NotReady before CNI

```
kubectl get nodes -o wide
kubectl get pods -n kube-system
```

Step 2 — Install Calico Operator

```
curl -O
https://raw.githubusercontent.com/projectcalico/calico/v3.29.0/manifests/
tigera-operator.yaml
curl -O
https://raw.githubusercontent.com/projectcalico/calico/v3.29.0/manifests/
custom-resources.yaml
kubectl apply -f tigera-operator.yaml
kubectl apply -f custom-resources.yaml
```

Step 3 — Watch nodes transition to Ready

```
kubectl get nodes -w
kubectl get pods -n calico-system
```

Step 4 — Test cross-node pod ping

```
kubectl run pod-cp --image=nicolaka/netshoot -- sleep infinity
kubectl run pod-worker --image=nicolaka/netshoot -- sleep infinity
kubectl get pods -o wide
```

```
POD_WORKER_IP=$(kubectl get pod pod-worker -o jsonpath='{.status.podIP}')
kubectl exec pod-cp -- ping -c 4 $POD_WORKER_IP
```

Note: Kubernetes requires a flat pod network -- cross-node pod-to-pod communication without NAT. A failed ping indicates CNI misconfiguration.

Test it: Both nodes are Ready. Cross-node ping returns 0% packet loss. coredns pods are Running.

Lab 4 — Cluster Upgrade: v1.34 to v1.35 with kubeadm drain/uncordon

Domain: Cluster Architecture, Installation & Configuration · Topic: Cluster upgrade · Duration: 60 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Goal

Upgrade a two-node cluster from v1.34 to v1.35 following the official kubeadm upgrade procedure. The exact sequence: unhold, install kubeadm, upgrade plan, upgrade apply, drain, upgrade kubelet/kubectl, uncordon. Missing any step is a common exam mistake.

What you'll build

Upgrade control plane with kubeadm upgrade apply, drain node, upgrade kubelet, uncordon. Repeat for worker.

Step 1 — Add v1.35 apt repository

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.35/deb/Release.key | \
  gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
apt-get update
```

Step 2 — Upgrade kubeadm on control plane

```
apt-mark unhold kubeadm
apt-get install -y kubeadm=1.35.0-*
apt-mark hold kubeadm
kubeadm version
```

Step 3 — Run upgrade plan and apply

```
kubeadm upgrade plan
kubeadm upgrade apply v1.35.0
```

Step 4 — Drain, upgrade kubelet, uncordon control plane

```
kubectl drain controlplane --ignore-daemonsets --delete-emptydir-data
apt-mark unhold kubelet kubectl
apt-get install -y kubelet=1.35.0-* kubectl=1.35.0-*
apt-mark hold kubelet kubectl
systemctl daemon-reload && systemctl restart kubelet
kubectl uncordon controlplane
```

Step 5 — Upgrade worker node

```
# On WORKER NODE:
apt-mark unhold kubeadm && apt-get install -y kubeadm=1.35.0-*
kubeadm upgrade node
# On CONTROL PLANE:
kubectl drain worker01 --ignore-daemonsets --delete-emptydir-data
# Back on WORKER:
apt-mark unhold kubelet kubectl
apt-get install -y kubelet=1.35.0-* kubectl=1.35.0-*
systemctl daemon-reload && systemctl restart kubelet
# On CONTROL PLANE:
kubectl uncordon worker01 && kubectl get nodes
```

Note: The VERSION column in 'kubectl get nodes' shows kubelet version, not API server version. systemctl daemon-reload is required after binary update before restarting kubelet.

Test it: kubectl get nodes shows both nodes as Ready at v1.35.0. kubectl version shows Server v1.35.0.

Lab 5 — HA Control Plane: HAProxy + Keepalived VIP and --upload-certs

Domain: Cluster Architecture, Installation & Configuration · Topic: High availability · Duration: 60 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Goal

Set up a highly available control plane topology using HAProxy as a TCP load balancer and Keepalived to provide a floating VIP. Initialize the first control plane with --control-plane-endpoint pointing to the VIP and --upload-certs to share certs. CKA tests etcd quorum and cluster resilience questions.

What you'll build

Configure HAProxy and Keepalived, run kubeadm init with --control-plane-endpoint and --upload-certs, join additional control plane nodes, verify etcd member list.

Step 1 — Install and configure HAProxy

```
apt-get install -y haproxy
systemctl restart haproxy && systemctl enable haproxy
```

Step 2 — Configure Keepalived with VIP

```
apt-get install -y keepalived
# MASTER: state MASTER, priority 101
# BACKUP: state BACKUP, priority 100
systemctl restart keepalived && systemctl enable keepalived
```

Step 3 — Initialize first control plane with VIP

```
kubeadm init \
  --kubernetes-version=v1.35.0 \
  --control-plane-endpoint='192.168.1.100:6443' \
  --pod-network-cidr=192.168.0.0/16 \
  --upload-certs
mkdir -p $HOME/.kube && cp /etc/kubernetes/admin.conf $HOME/.kube/config
```

Note: --control-plane-endpoint must point to the VIP address and is baked into all certs/kubeconfigs. --upload-certs stores encrypted certs in a Secret. The --certificate-key expires after 2 hours.

Step 4 — Join additional control plane nodes

```
kubeadm join 192.168.1.100:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>
```

Step 5 — Verify etcd HA cluster

```
kubectl -n kube-system exec -it etcd-control-plane-01 -- \
  etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key member list
```

Test it: etcdctl member list shows all control plane nodes as healthy. kubectl get nodes shows 3+ nodes. VIP migrates to secondary when primary API server is stopped.

Lab 6 — Helm: Install, Upgrade, Rollback, --set, -f values.yaml, helm template

Domain: Cluster Architecture, Installation & Configuration · Topic: Helm package manager · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Master the complete Helm workflow: adding repos, installing charts, overriding values with --set and -f, upgrading releases, rollbacks, and using helm template to render manifests. Helm is explicitly allowed as a reference during the CKA exam at helm.sh/docs.

What you'll build

Add bitnami repo, install nginx chart with custom values, upgrade, rollback, use helm template for GitOps-style rendering.

Step 1 — Install Helm and add repos

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
```

Step 2 — Install a chart with value overrides

```
helm install my-nginx bitnami/nginx \
  --namespace web --create-namespace \
  --set service.type=NodePort \
  --set replicaCount=2
helm list -n web
helm status my-nginx -n web
```

Step 3 — Upgrade with -f values.yaml

```
cat <<'EOF' > my-values.yaml
replicaCount: 3
service:
  type: ClusterIP
EOF
helm upgrade my-nginx bitnami/nginx -n web -f my-values.yaml
helm history my-nginx -n web
```

Step 4 — Roll back a release

```
helm rollback my-nginx -n web
helm history my-nginx -n web
```

Step 5 — Render manifests without installing

```
helm template my-nginx bitnami/nginx --set replicaCount=2
helm template my-nginx bitnami/nginx -f my-values.yaml > rendered.yaml
```

Note: helm template is used in GitOps workflows. -f values override defaults; --set overrides -f when both are specified.

Test it: helm list -n web shows my-nginx deployed. helm history shows at least 2 revisions. helm rollback reverts to the previous version.

Lab 7 — Kustomize: Base + Overlays, namePrefix, images, patches, kubectl apply -k

Domain: Cluster Architecture, Installation & Configuration · Topic: Kustomize · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Use Kustomize (built into kubectl) to manage base manifests and environment-specific overlays without file duplication. CKA tests kubectl apply -k, namePrefix, image overrides, and strategic merge patches.

What you'll build

Create base (deployment + service + kustomization.yaml), dev overlay with namePrefix and image override, production overlay with replica patch. Apply with kubectl apply -k.

Step 1 — Verify kustomize is built into kubectl

```
kubectl kustomize --help
kubectl version --client -o yaml | grep -i kustomize
```

Step 2 — Create base layer

```
mkdir -p ~/kustomize-lab/base ~/kustomize-lab/overlays/dev
# Create base/deployment.yaml, base/service.yaml
# Create base/kustomization.yaml with resources list
kubectl kustomize ~/kustomize-lab/base
```

Step 3 — Dev overlay with namePrefix

```
cat <<'EOF' > ~/kustomize-lab/overlays/dev/kustomization.yaml
bases:
  - ../../base
namePrefix: dev-
namespace: dev
```

```
images:
  - name: nginx
    newTag: '1.25-alpine'
EOF
kubectl kustomize ~/kustomize-lab/overlays/dev
```

Step 4 — Apply overlays

```
kubectl create namespace dev
kubectl apply -k ~/kustomize-lab/overlays/dev
kubectl get deployment -n dev
# NAME: dev-webapp
```

Step 5 — Preview changes before applying

```
kubectl diff -k ~/kustomize-lab/overlays/dev
```

Note: namePrefix automatically updates all cross-references (selectors, labels). kubectl diff -k previews changes before applying -- essential for safe production.

Test it: kubectl get deployment -n dev shows dev-webapp (namePrefix applied). kubectl diff -k shows planned changes without modifying the cluster.

Lab 8 — RBAC: Roles, RoleBindings, ServiceAccounts, ClusterRole

Domain: Cluster Architecture, Installation & Configuration · Topic: RBAC · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Create a ServiceAccount for a read-only viewer persona, define a namespaced Role that allows only get/list/watch on Pods, bind the role to the SA, and prove that the SA can read but cannot write. Then extend to ClusterRole for cluster-scoped resources.

What you'll build

Create SA + Role + RoleBinding in rbac-demo namespace, test with kubectl auth can-i --as, verify from inside a pod, add ClusterRole for node access.

Step 1 — Create namespace and ServiceAccount

```
kubectl create ns rbac-demo
kubectl -n rbac-demo create serviceaccount viewer
```

Step 2 — Create a namespaced Role

```
kubectl -n rbac-demo create role pod-viewer \
```

```
  --verb=get,list,watch \
  --resource=pods
kubectl -n rbac-demo get role pod-viewer -o yaml
```

Step 3 — Bind Role to ServiceAccount

```
kubectl -n rbac-demo create rolebinding viewer-binding \
  --role=pod-viewer \
  --serviceaccount=rbac-demo:viewer
```

Step 4 — Test with kubectl auth can-i

```
kubectl -n rbac-demo auth can-i list pods \
  --as=system:serviceaccount:rbac-demo:viewer
kubectl -n rbac-demo auth can-i create pods \
  --as=system:serviceaccount:rbac-demo:viewer
# Expected: yes, no
```

Step 5 — ClusterRole for cluster-scoped resources

```
kubectl create clusterrole node-reader --verb=get,list,watch --
resource=nodes
kubectl create clusterrolebinding viewer-nodes \
  --clusterrole=node-reader \
  --serviceaccount=rbac-demo:viewer
```

Note: Identity format for SA: system:serviceaccount:<namespace>:<name>. ClusterRole is for cluster-scoped resources (nodes, PVs); Role is namespace-scoped.

Test it: kubectl auth can-i list pods --as=SA returns yes; create pods returns no. ClusterRole allows viewer SA to list nodes across all namespaces.

DAY 2 — DOMAINS 1+2+3: Workloads, Scheduling & Services (Labs 9–18)

Lab 9 — CRDs and Operators: Schema Validation, cert-manager, Certificate Lifecycle

Domain: Cluster Architecture, Installation & Configuration · Topic: CRDs and Operators · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Define a CRD with OpenAPI schema validation, create custom resources, install the cert-manager operator via Helm, and walk through the TLS certificate lifecycle. CKA tests working with CRDs and operator-managed resources.

What you'll build

Create AppDatabase CRD with enum/pattern/required validation, install cert-manager, create self-signed ClusterIssuer, request TLS Certificate, inspect resulting Secret.

Step 1 — Define a CRD with schema validation

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: appdatabases.storage.example.com
spec:
  group: storage.example.com
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            required: ['engine', 'storage']
            properties:
              engine:
                type: string
                enum: ['postgresql', 'mysql', 'redis']
  scope: Namespaced
  names:
    plural: appdatabases
    singular: appdatabase
    kind: AppDatabase
EOF
```

Step 2 — Create and query custom resources

```
kubectl apply -f - <<'EOF'
apiVersion: storage.example.com/v1
kind: AppDatabase
metadata:
  name: my-postgres
spec:
  engine: postgresql
  storage: 20Gi
EOF
kubectl get appdatabases
kubectl describe adb my-postgres
```

Step 3 — Install cert-manager via Helm

```
helm repo add jetstack https://charts.jetstack.io && helm repo update
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager --create-namespace \
  --version v1.15.0 --set installCRDs=true
kubectl get pods -n cert-manager
```

Step 4 — Create ClusterIssuer and request certificate

```
kubectl apply -f - <<'EOF'
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: selfsigned-issuer
spec:
  selfSigned: {}
EOF
kubectl wait --for=condition=Ready clusterissuer/selfsigned-issuer
```

Note: CRDs extend the Kubernetes API with custom resource types. OpenAPI schema validation rejects invalid resources at the API server before they reach etcd.

Test it: kubectl get appdatabases shows my-postgres. cert-manager pods are Running. ClusterIssuer is Ready. Invalid engine value (e.g. 'oracle') is rejected by the API server.

Lab 10 — Extension Interfaces: CRI (crictl), CNI (/etc/cni/net.d), CSI (kubectl get csidrivers)

Domain: Cluster Architecture, Installation & Configuration · Topic: CRI/CNI/CSI extension interfaces · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Explore the three Kubernetes extension interfaces using native debugging tools: crictl for CRI, config file inspection for CNI, and kubectl for CSI drivers. Critical for troubleshooting pods stuck in ContainerCreating or volume mount failures.

What you'll build

Configure crictl.yaml, list containers/images with crictl, inspect CNI config, list CSI drivers, trace a pod network setup with veth pairs.

Step 1 — Configure and use crictl

```
cat <<'EOF' > /etc/crictl.yaml
```

```
runtime-endpoint: unix:///run/containerd/containerd.sock
image-endpoint: unix:///run/containerd/containerd.sock
timeout: 10
EOF
crictl info && crictl pods && crictl ps && crictl images
```

Step 2 — Get container logs with crictl

```
POD_ID=$(crictl pods --name coredns -q | head -1)
CONTAINER_ID=$(crictl ps --pod $POD_ID -q | head -1)
crictl logs --tail=20 $CONTAINER_ID
```

Step 3 — Inspect CNI configuration

```
ls -la /etc/cni/net.d/
cat /etc/cni/net.d/10-calico.conflist
ls /opt/cni/bin/
```

Step 4 — Inspect CSI drivers

```
kubectl get csidrivers
kubectl get csinodes
kubectl get volumeattachments
```

Note: crictl logs works even when kubectl is unavailable (API server down). CNI config: lowest-numbered file in /etc/cni/net.d/ is used. Missing CNI binary in /opt/cni/bin/ causes ContainerCreating to hang indefinitely.

Test it: crictl pods lists all running pods. CNI config file exists in /etc/cni/net.d/. kubectl get csidrivers returns available storage drivers.

Lab 11 — Deployments and Rollouts: Rolling Update, Bad Image, rollout undo, pause/resume

Domain: Workloads & Scheduling · Topic: Deployments and rollouts · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Create a Deployment, perform a rolling update, deliberately introduce a broken image to trigger a failed rollout, roll back using kubectl rollout undo, and use pause/resume to batch multiple changes. Mastering kubectl rollout subcommands is essential for the 15% Workloads & Scheduling domain.

What you'll build

Create 4-replica nginx Deployment, update to new image, break it with invalid tag, rollback, roll back to specific revision, pause/resume.

Step 1 — Create and expose Deployment

```
kubectl create deployment webapp --image=nginx:1.25 --replicas=4 --port=80
kubectl expose deployment webapp --port=80
kubectl rollout history deployment webapp
```

Step 2 — Perform a rolling update

```
kubectl set image deployment/webapp webapp=nginx:1.27
kubectl annotate deployment webapp kubernetes.io/change-cause='upgrade to
nginx:1.27'
kubectl rollout status deployment webapp
```

Step 3 — Simulate a broken rollout

```
kubectl set image deployment/webapp webapp=nginx:does-not-exist-999
kubectl get pods -l app=webapp
# Old pods Running; new pod ImagePullBackOff
```

Step 4 — Roll back

```
kubectl rollout undo deployment webapp
kubectl rollout status deployment webapp
kubectl rollout undo deployment webapp --to-revision=1
```

Step 5 — Pause and resume a rollout

```
kubectl set image deployment/webapp webapp=nginx:1.27
kubectl rollout pause deployment webapp
kubectl set resources deployment webapp --containers=webapp --
requests=cpu=100m
kubectl rollout resume deployment webapp
kubectl rollout status deployment webapp
```

Note: Rolling updates are safe: broken image stalls rollout without taking down healthy pods. kubectl rollout undo creates a NEW revision -- the counter always grows.

Test it: kubectl rollout history shows multiple revisions. After rollback all pods run previous image. Service continues responding during the bad image period.

Lab 12 — ConfigMaps: 3 Creation Methods, envFrom, Volume Mount, Live Update Semantics

Domain: Workloads & Scheduling · Topic: ConfigMaps · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Create ConfigMaps using three methods (literal, file, YAML manifest), inject via envFrom and env.valueFrom, mount as volumes with custom filenames and permissions, and demonstrate the live-update behavior difference. Understanding live vs static updates is a tested CKA concept.

What you'll build

Create app-config from literals, file-config from files, manifest-config from YAML. Test envFrom with prefix, selective keys, volume mount, subPath, and live update.

Step 1 — Create ConfigMap three ways

```
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info
kubectl create configmap file-config --from-file=/tmp/nginx.conf
kubectl get configmap app-config -o yaml
```

Step 2 — Inject with envFrom

```
# Pod spec:
envFrom:
- configMapRef:
    name: app-config
  prefix: 'CONFIG_'
kubectl exec envfrom-pod -- env | grep CONFIG_
```

Step 3 — Mount as volume

```
# Pod spec:
volumeMounts:
- name: config-vol
  mountPath: /config
volumes:
- name: config-vol
  configMap:
    name: app-config
kubectl exec volume-mount-pod -- ls /config
kubectl exec volume-mount-pod -- cat /config/LOG_LEVEL
```

Step 4 — Live update semantics

```
kubectl patch configmap app-config --type=merge -p '{"data":
{"LOG_LEVEL":"debug"}}'
```

```
sleep 90
kubectl exec volume-mount-pod -- cat /config/LOG_LEVEL
# debug -- updated live!
kubectl exec envfrom-pod -- env | grep LOG_LEVEL
# info -- NOT updated
```

Note: CRITICAL: Volume-mounted ConfigMaps update automatically (1-2 min delay). Env vars and subPath mounts require pod restart to pick up changes.

Test it: volume-mount-pod shows updated LOG_LEVEL=debug after ConfigMap patch. envfrom-pod still shows LOG_LEVEL=info (not live-updated).

Lab 13 — Secrets: generic/tls/docker-registry, Env Inject, tmpfs Mount defaultMode 0400

Domain: Workloads & Scheduling · Topic: Secrets · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Create secrets of three types (generic, TLS, docker-registry), inject via environment variables and volume mounts backed by tmpfs, and apply restrictive permissions with defaultMode 0400. Understand that Secrets are base64-encoded but NOT encrypted by default.

What you'll build

Create db-credentials (generic), webapp-tls (TLS with openssl), registry secret. Mount with defaultMode 0400, verify tmpfs, decode with base64 -d.

Step 1 — Create and decode generic secret

```
kubectl create secret generic db-credentials \
  --from-literal=DB_USER=admin \
  --from-literal=DB_PASSWORD=SuperSecret123!
kubectl get secret db-credentials -o yaml
kubectl get secret db-credentials \
  -o jsonpath='{.data.DB_PASSWORD}' | base64 -d
```

Step 2 — Create TLS secret

```
openssl req -x509 -nodes -newkey rsa:2048 \
  -keyout /tmp/tls.key -out /tmp/tls.crt -days 365 \
  -subj '/CN=webapp.example.com'
kubectl create secret tls webapp-tls --cert=/tmp/tls.crt --key=/tmp/tls.key
```

Step 3 — Mount as tmpfs with defaultMode 0400

```
# Pod spec volumes:
volumes:
- name: secret-vol
  secret:
    secretName: db-credentials
    defaultMode: 0400
kubectl exec secret-volume-pod -- ls -la /secrets
# -r-------- files
kubectl exec secret-volume-pod -- df -h /secrets
# tmpfs
```

Note: Secrets are base64-encoded (NOT encrypted). Enable etcd encryption at rest for actual security. tmpfs means secrets are never written to node disk. echo -n is critical -- without -n, trailing newline corrupts the base64 value.

Test it: Secret files have permissions -r-------- (0400). df -h /secrets shows tmpfs. base64 -d decodes the password correctly. TLS secret type is kubernetes.io/tls.

Lab 14 — HPA v2: metrics-server, HorizontalPodAutoscaler, Load Test, Watch Scale

Domain: Workloads & Scheduling · Topic: Horizontal Pod Autoscaling · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Install metrics-server to provide CPU metrics, create an HPA using autoscaling/v2 API, generate CPU load, and watch the HPA scale up then back down. HPA requires resource requests on containers; without them, utilization cannot be calculated.

What you'll build

Install metrics-server (with --kubelet-insecure-tls patch), create HPA targeting 50% CPU, run stress test, watch scale-up, wait for scale-down.

Step 1 — Install metrics-server

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml
kubectl patch deployment metrics-server -n kube-system --type=json \

-p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"-
-kubelet-insecure-tls"}]'
until kubectl top node 2>/dev/null; do echo 'waiting...'; sleep 5; done
```

Step 2 — Deployment with resource requests

```
kubectl create deployment cpu-app --image=nginx --replicas=1
kubectl set resources deployment cpu-app --containers=nginx \
  --requests=cpu=100m --limits=cpu=500m
```

Step 3 — Create HPA

```
kubectl autoscale deployment cpu-app --min=1 --max=10 --cpu-percent=50
kubectl get hpa && kubectl describe hpa cpu-app
```

Step 4 — Generate load and watch scaling

```
kubectl run load-gen --image=busybox --restart=Never \
  -- sh -c 'while true; do wget -q -O- http://cpu-app; done'
kubectl get hpa cpu-app -w
kubectl get pods -w
```

Note: HPA requires resource requests -- without them metrics-server cannot calculate % utilization. Scale-down has a 5-minute cooldown by default.

Test it: kubectl get hpa shows non-zero CPU% and increasing replicas under load. After removing load-gen, replicas scale back to 1 within 5-10 minutes.

Lab 15 — Self-Healing: Liveness/Readiness Probes, ReplicaSet, DaemonSet, StatefulSet

Domain: Workloads & Scheduling · Topic: Self-healing workloads · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Configure liveness and readiness probes to control container health, demonstrate ReplicaSet self-healing when pods are manually deleted, deploy a DaemonSet that runs one pod per node, and create a StatefulSet with stable network identity.

What you'll build

Deploy pod with httpGet liveness probe, show ReplicaSet maintains desired count, deploy nginx DaemonSet, create StatefulSet with volumeClaimTemplates.

Step 1 — Liveness probe: httpGet

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: liveness-http
```

```
spec:
  containers:
  - name: webapp
    image: nginx:alpine
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 10
      failureThreshold: 3
EOF
kubectl get pod liveness-http -w
```

Step 2 — Readiness probe blocks traffic

```
# Readiness probe -- pod not added to Service endpoints until probe passes
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 10
```

Step 3 — ReplicaSet self-healing

```
kubectl create deployment selfheal --image=nginx --replicas=3
kubectl delete pod -l app=selfheal --force
kubectl get pods -l app=selfheal -w
# New pods created immediately
```

Step 4 — DaemonSet: one pod per node

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-monitor
spec:
  selector:
    matchLabels:
      app: node-monitor
  template:
    metadata:
      labels:
        app: node-monitor
    spec:
      containers:
      - name: monitor
        image: busybox
        command: ['sleep', '3600']
EOF
kubectl get daemonset
kubectl get pods -l app=node-monitor -o wide
```

Note: Liveness: fail -> container restart. Readiness: fail -> removed from Service endpoints. DaemonSet automatically places one pod per node, including newly joined nodes.

Test it: Pod restarts when liveness probe fails. kubectl delete pod does not reduce ReplicaSet count. DaemonSet has exactly one pod per node.

Lab 16 — Scheduling: nodeSelector, Node Affinity, Taints + Tolerations, Resource Requests

Domain: Workloads & Scheduling · Topic: Scheduling · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Control pod placement using nodeSelector for simple label matching, node affinity for flexible required/preferred placement rules, taints to repel pods from nodes, and tolerations to allow specific pods through taints.

What you'll build

Label nodes, create pod with nodeSelector, add required/preferred node affinity, taint a node (NoSchedule), add toleration, demonstrate resource-based scheduling.

Step 1 — nodeSelector: simple label matching

```
kubectl label node worker01 disk=ssd
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: ssd-pod
spec:
  nodeSelector:
    disk: ssd
  containers:
  - name: app
    image: nginx
EOF
kubectl get pod ssd-pod -o wide
```

Step 2 — Node Affinity

```
# requiredDuringSchedulingIgnoredDuringExecution: hard constraint
# preferredDuringSchedulingIgnoredDuringExecution: soft preference
# Check: kubectl get pod affinity-pod -o wide
```

Step 3 — Taints and Tolerations

```
kubectl taint node worker01 special=gpu:NoSchedule
kubectl run blocked --image=nginx
kubectl get pod blocked -o wide
# Stays Pending -- no toleration
# Add toleration:
tolerations:
- key: special
  operator: Equal
  value: gpu
  effect: NoSchedule
```

Step 4 — Remove taint and clean up

```
kubectl taint node worker01 special=gpu:NoSchedule-
kubectl get pods
```

Note: Taint effects: NoSchedule (hard reject), PreferNoSchedule (soft reject), NoExecute (evict existing pods). Tolerations do NOT guarantee placement on the tainted node.

Test it: nodeSelector pod lands on worker01 (disk=ssd). Tainted node rejects pods without toleration. Pod with matching toleration schedules on the tainted node.

Lab 17 — Pod Connectivity: Flat Pod Network, ping, curl, DNS Discovery, netshoot

Domain: Services & Networking · Topic: Pod networking · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Verify pod-to-pod communication using ping and curl, test DNS-based service discovery, trace network paths with traceroute, and use nicolaka/netshoot as a network debugging toolkit. Pod networking fundamentals are the foundation for the 20% Services & Networking domain.

What you'll build

Deploy server-pod and client-pod, ping by Pod IP, curl by Service DNS name, nslookup kubernetes.default, traceroute between pods.

Step 1 — Deploy test pods

```
kubectl run server-pod --image=nginx --labels='role=server'
kubectl expose pod server-pod --port=80 --name=server-svc
kubectl run client-pod --image=nicolaka/netshoot --command -- sleep 3600
kubectl get pods -o wide
```

Step 2 — Ping by Pod IP

```
SERVER_IP=$(kubectl get pod server-pod -o jsonpath='{.status.podIP}')
kubectl exec client-pod -- ping -c 4 $SERVER_IP
```

Step 3 — DNS-based service discovery

```
kubectl exec client-pod -- curl -s http://server-
svc.default.svc.cluster.local
kubectl exec client-pod -- nslookup kubernetes.default.svc.cluster.local
kubectl exec client-pod -- cat /etc/resolv.conf
```

Step 4 — Network debugging with netshoot

```
kubectl exec client-pod -- traceroute $SERVER_IP
kubectl exec client-pod -- dig server-svc.default.svc.cluster.local
kubectl exec client-pod -- ss -tulpn
```

Note: DNS pattern: <service>.<namespace>.svc.cluster.local. nicolaka/netshoot includes ping, curl, dig, traceroute, nmap, tshark, tcpdump.

Test it: Ping to server-pod IP returns 0% loss. curl to service DNS returns nginx HTML. nslookup kubernetes.default resolves to the ClusterIP of the kubernetes service.

Lab 18 — Service Types: ClusterIP, NodePort, LoadBalancer (MetalLB), EndpointSlices, Selector Debug

Domain: Services & Networking · Topic: Service types · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Create and test all three primary service types -- ClusterIP, NodePort, and LoadBalancer with MetalLB -- inspect EndpointSlice objects, and debug selector mismatches that cause services to have no endpoints. Service types are heavily tested (20% CKA domain).

What you'll build

Create ClusterIP, NodePort with custom port, inspect EndpointSlices, create service with wrong selector and debug/fix it.

Step 1 — ClusterIP service

```
kubectl create deployment webapp --image=nginx --replicas=3
kubectl expose deployment webapp --port=80 --type=ClusterIP
```

```
kubectl get svc webapp
kubectl get endpointslices -l kubernetes.io/service-name=webapp
```

Step 2 — NodePort service

```
kubectl expose deployment webapp --port=80 --type=NodePort --name=webapp-np
NODE_PORT=$(kubectl get svc webapp-np -o
jsonpath='{.spec.ports[0].nodePort}')
echo "NodePort: $NODE_PORT"
```

Step 3 — Debug: selector mismatch causing no endpoints

```
# Create service with wrong selector
kubectl expose deployment webapp --port=80 --name=broken-svc
kubectl patch svc broken-svc -p '{"spec":{"selector":{"app":"wrong"}}}'
kubectl get endpoints broken-svc
# <none> -- no matching pods
# Fix: correct the selector
kubectl patch svc broken-svc -p '{"spec":{"selector":{"app":"webapp"}}}'
```

Note: EndpointSlices replace Endpoints in Kubernetes v1.21+. A service with no endpoints usually means a selector label mismatch. Compare: kubectl describe svc vs kubectl get pods --show-labels.

Test it: ClusterIP routes traffic to all 3 pods. NodePort is accessible from node IP. broken-svc has no endpoints; fixing selector restores them immediately.

DAY 3 — DOMAINS 3+4+5: Networking, Storage & Troubleshooting (Labs 19–28)

Lab 19 — Ingress Controller: Host Routing, TLS Termination, Path Rules, Debug

Domain: Services & Networking · Topic: Ingress · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Install ingress-nginx controller, configure host-based routing to multiple backends, terminate TLS using a Kubernetes Secret, and debug misconfigured Ingress resources. CKA 2026 tests installing ingress-nginx, host routing, and TLS termination.

What you'll build

Install ingress-nginx via bare-metal manifest, create Ingress for two apps with host rules, add TLS block with secret, debug 404 by checking ingressClassName.

Step 1 — Install ingress-nginx

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-
nginx/main/deploy/static/provider/baremetal/deploy.yaml
kubectl -n ingress-nginx wait --for=condition=Ready pod \
  -l app.kubernetes.io/component=controller --timeout=180s
```

Step 2 — Get NodePort for testing

```
HTTP_PORT=$(kubectl -n ingress-nginx get svc ingress-nginx-controller \
  -o jsonpath='{.spec.ports[?(@.name=="http")].nodePort}')
echo "HTTP NodePort: $HTTP_PORT"
```

Step 3 — Create Ingress with host-based routing

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multi-host
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: app1.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app1-svc
            port:
              number: 80
EOF
```

Step 4 — TLS termination

```
kubectl create secret tls tls-secret --cert=tls.crt --key=tls.key
# Add to Ingress spec:
# tls:
# - hosts: [app1.example.com]
#   secretName: tls-secret
```

Note: ingressClassName: nginx is required in Kubernetes 1.18+. Without it, the Ingress is ignored. Check ingress-nginx logs: kubectl logs -n ingress-nginx -l app.kubernetes.io/component=controller

Test it: curl -H 'Host: app1.example.com' http://NODE:PORT returns app1 content. Wrong ingressClassName causes 404. TLS ingress terminates HTTPS correctly.

Lab 20 — Gateway API: GatewayClass, Gateway, HTTPRoute (GA in v1.28, tested in CKA 2026)

Domain: Services & Networking · Topic: Gateway API · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Use the Kubernetes Gateway API (GA as of v1.28, tested in CKA 2026) to route traffic using GatewayClass, Gateway, and HTTPRoute objects. Gateway API is role-oriented: infrastructure providers manage GatewayClass/Gateway; developers manage HTTPRoutes.

What you'll build

Install Gateway API CRDs and NGINX Gateway Fabric, create GatewayClass and Gateway, define HTTPRoutes with path-based routing.

Step 1 — Install Gateway API CRDs

```
kubectl apply -f
https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.1.0/
standard-install.yaml
kubectl get crds | grep gateway
```

Step 2 — Install NGINX Gateway Fabric

```
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-
fabric/v1.4.0/deploy/crds.yaml
kubectl apply -f https://raw.githubusercontent.com/nginxinc/nginx-gateway-
fabric/v1.4.0/deploy/manifests/nginx-gateway.yaml
```

Step 3 — Create GatewayClass and Gateway

```
cat <<'EOF' | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx
spec:
  controllerName: gateway.nginx.org/nginx-gateway-controller
---
```

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: main-gateway
spec:
  gatewayClassName: nginx
  listeners:
  - name: http
    port: 80
    protocol: HTTP
EOF
```

Step 4 — Create HTTPRoute

```
cat <<'EOF' | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: app-route
spec:
  parentRefs:
  - name: main-gateway
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /api
    backendRefs:
    - name: api-svc
      port: 80
EOF
```

Note: Three core Gateway API objects: GatewayClass (infra tier), Gateway (cluster operator tier), HTTPRoute (developer tier). Role separation enables multi-tenant traffic management.

Test it: kubectl get gatewayclass shows nginx as Accepted. kubectl get gateway shows Programmed=True. HTTP request to /api path routes to api-svc backend.

Lab 21 — NetworkPolicy: Default-Deny, Pod Selector, Namespace Selector, Egress Lockdown

Domain: Services & Networking · Topic: Network Policies · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Implement Pod-level firewall rules using NetworkPolicies. Kubernetes uses allow-lists: once any NetworkPolicy selects a pod, all traffic not explicitly allowed is blocked. CKA 2026 tests default-deny, pod/namespace selectors, and egress lockdown.

What you'll build

Create server and client pods, apply default-deny-ingress policy, verify blocking, add allow rule for specific pod selector, test egress lockdown.

Step 1 — Create test workloads

```
kubectl create ns netpol
kubectl -n netpol run server --image=nginx --labels='app=server'
kubectl -n netpol run client-ok --image=nicolaka/netshoot \
  --labels='role=allowed' --command -- sleep 3600
kubectl -n netpol run client-bad --image=nicolaka/netshoot \
  --labels='role=denied' --command -- sleep 3600
kubectl -n netpol expose pod server --port=80
```

Step 2 — Apply default-deny-ingress

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: netpol
spec:
  podSelector: {}
  policyTypes: ['Ingress']
EOF
kubectl -n netpol exec client-ok -- curl -m 3 http://server || echo BLOCKED
```

Step 3 — Allow specific pod selector

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-allowed
  namespace: netpol
spec:
  podSelector:
    matchLabels:
      app: server
  policyTypes: ['Ingress']
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: allowed
EOF
```

```
kubectl -n netpol exec client-ok -- curl -s http://server
# OK
kubectl -n netpol exec client-bad -- curl -m 3 http://server || echo BLOCKED
```

Note: Empty podSelector {} selects ALL pods in the namespace. NetworkPolicy requires a CNI that enforces it (Calico, Cilium -- NOT Flannel alone).

Test it: default-deny blocks both clients. After allow policy, client-ok reaches server; client-bad (role=denied) is still blocked.

Lab 22 — CoreDNS: Corefile Inspection, Service DNS, Headless Services, Stub Zones

Domain: Services & Networking · Topic: CoreDNS · Duration: 30 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Inspect CoreDNS configuration, query Service and Pod DNS records, understand headless service resolution (A records per pod), and customize CoreDNS with stub zones. CKA tests configuring CoreDNS via ConfigMap and diagnosing DNS failures.

What you'll build

Inspect kube-dns service and coredns ConfigMap, test DNS from netshoot pod, create headless service, verify per-pod A records, add stub zone to Corefile.

Step 1 — Inspect CoreDNS

```
kubectl -n kube-system get svc kube-dns
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl -n kube-system get configmap coredns -o yaml
```

Step 2 — Query DNS from debug pod

```
kubectl run dnsdebug --image=nicolaka/netshoot --command -- sleep 3600
kubectl exec dnsdebug -- cat /etc/resolv.conf
kubectl exec dnsdebug -- nslookup kubernetes.default.svc.cluster.local
kubectl exec dnsdebug -- dig +short webapp.default.svc.cluster.local
```

Step 3 — Headless service: per-pod A records

```
kubectl expose deployment webapp --port=80 --name=webapp-headless --cluster-ip=None
kubectl exec dnsdebug -- dig +short webapp-headless.default.svc.cluster.local
# Returns one IP per pod -- used by StatefulSets for stable identity
```

Step 4 — Add stub zone to Corefile

```
kubectl -n kube-system edit configmap coredns
# Add inside Corefile:
# example.internal:53 {
#   forward . 10.10.0.1
# }
```

Note: Service name is 'kube-dns' (legacy) even though pods run CoreDNS. Headless services have clusterIP: None. Stub zones forward specific domains to external DNS.

Test it: nslookup kubernetes.default resolves to 10.96.0.1. Headless service dig returns one IP per pod. CoreDNS ConfigMap has the Corefile visible.

Lab 23 — PersistentVolume and PVC: Static Provisioning, Access Modes, Reclaim Policies

Domain: Storage · Topic: PersistentVolumes and PersistentVolumeClaims · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Create a PersistentVolume (static provisioning), bind it via a PVC, mount in a pod, and understand PV/PVC binding rules including access modes and reclaim policies. CKA tests static provisioning, access modes, and the fact that PVCs outlive pods.

What you'll build

Create hostPath PV with ReadWriteOnce, create PVC that binds, mount in nginx pod, write data, delete pod and recreate to verify persistence.

Step 1 — Create a PersistentVolume

```
sudo mkdir -p /mnt/data && echo 'hello from host' | sudo tee
/mnt/data/index.html
cat > pv.yaml <<'EOF'
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-host
spec:
  capacity:
    storage: 1Gi
  accessModes: [ReadWriteOnce]
  hostPath:
    path: /mnt/data
  persistentVolumeReclaimPolicy: Retain
EOF
```

```
kubectl apply -f pv.yaml && kubectl get pv
```

Step 2 — Create a PVC that binds to the PV

```
cat > pvc.yaml <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-host
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
EOF
kubectl apply -f pvc.yaml && kubectl get pvc
# STATUS: Bound
```

Step 3 — Mount PVC in a pod

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: pvc-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
    - name: data
      mountPath: /usr/share/nginx/html
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: pvc-host
EOF
kubectl exec pvc-pod -- cat /usr/share/nginx/html/index.html
```

Note: Access modes: ReadWriteOnce (one node RW), ReadOnlyMany (many nodes R), ReadWriteMany (many nodes RW). Reclaim: Retain (manual cleanup), Delete (removes storage), Recycle (deprecated).

Test it: PVC shows STATUS=Bound. Pod reads 'hello from host' from the mounted volume. After pod deletion and recreation, data persists (Retain policy).

Lab 24 — StorageClass: Dynamic Provisioning, Default Class, StatefulSet volumeClaimTemplates

Domain: Storage · Topic: StorageClass and dynamic provisioning · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Use StorageClasses for dynamic PV creation, mark a class as default, and create a StatefulSet with volumeClaimTemplates that provisions one PVC per pod. CKA tests creating StorageClasses and dynamic provisioning.

What you'll build

Install local-path-provisioner, mark StorageClass as default, create dynamically-provisioned PVC, create StatefulSet with volumeClaimTemplates.

Step 1 — Install local-path-provisioner

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-storage.yaml
kubectl -n local-path-storage rollout status deploy/local-path-provisioner
kubectl get storageclass
```

Step 2 — Mark StorageClass as default

```
kubectl patch storageclass local-path \
  -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

Step 3 — Dynamic PVC provisioning

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dynamic-pvc
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: local-path
  resources:
    requests:
      storage: 512Mi
EOF
kubectl get pvc dynamic-pvc && kubectl get pv
```

Step 4 — StatefulSet with volumeClaimTemplates

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: StatefulSet
metadata:
```

```
  name: db
spec:
  serviceName: db
  replicas: 3
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
      - name: postgres
        image: postgres:16-alpine
        env:
        - name: POSTGRES_PASSWORD
          value: example
        volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ReadWriteOnce]
      resources:
        requests:
          storage: 1Gi
EOF
kubectl get pods -l app=db && kubectl get pvc -l app=db
```

Note: StatefulSet volumeClaimTemplates create one PVC per pod: data-db-0, data-db-1, data-db-2. PVCs are NOT deleted when the StatefulSet is deleted -- manual cleanup required.

Test it: dynamic-pvc is Bound. StatefulSet db has 3 pods each with their own PVC (data-db-0, etc).

Lab 25 — Volume Types: emptyDir, hostPath, projected, downwardAPI

Domain: Storage · Topic: Pod volume types · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Explore pod-level volume types: emptyDir for shared scratch space, hostPath for node filesystem access, projected for combining multiple sources into one mount, and downwardAPI for exposing pod metadata to the application.

What you'll build

Create pod with emptyDir shared between two containers, projected volume combining ConfigMap + Secret, downwardAPI volume exposing pod name/namespace/resource requests.

Step 1 — emptyDir: shared scratch space

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: scratch
spec:
  containers:
  - name: writer
    image: busybox
    command: ['sh', '-c', 'echo hello-shared > /data/file; sleep 3600']
    volumeMounts:
    - name: tmp
      mountPath: /data
  - name: reader
    image: busybox
    command: ['sh', '-c', 'sleep 3600']
    volumeMounts:
    - name: tmp
      mountPath: /data
  volumes:
  - name: tmp
    emptyDir: {}
EOF
kubectl exec scratch -c reader -- cat /data/file
```

Step 2 — projected volume (ConfigMap + Secret)

```
# volumes:
# - name: combined
#   projected:
#     sources:
#     - configMap:
#         name: app-config
#     - secret:
#         name: app-secret
```

Step 3 — downwardAPI: pod metadata as files

```
# volumes:
# - name: podinfo
```

```
#   downwardAPI:
#     items:
#     - path: 'pod-name'
#       fieldRef:
#         fieldPath: metadata.name
#     - path: 'cpu-request'
#       resourceFieldRef:
#         resource: requests.cpu
kubectl exec downward-pod -- cat /podinfo/pod-name
```

Note: emptyDir is deleted when the pod is deleted. hostPath ties the pod to a specific node. projected combines ConfigMap, Secret, serviceAccountToken, and downwardAPI into one mount.

Test it: reader container reads file written by writer via emptyDir. downwardAPI pod shows correct pod name from /podinfo/pod-name.

Lab 26 — Troubleshoot Cluster Components: API Server, Controller Manager, Scheduler, etcd

Domain: Troubleshooting · Topic: Control plane troubleshooting · Duration: 60 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Goal

Diagnose and fix broken control plane components. When kubectl is unresponsive, use crictl and journalctl instead. Troubleshooting is the highest-weighted CKA domain at 30%.

What you'll build

Locate static pod manifests, introduce a break in kube-apiserver.yaml (wrong flag), diagnose with crictl and journalctl, fix manifest and verify recovery.

Step 1 — Know where control plane components live

```
ls /etc/kubernetes/manifests/
sudo systemctl status kubelet --no-pager | head -5
sudo systemctl status containerd --no-pager | head -5
```

Step 2 — Diagnose API server failure

```
# If kubectl is unresponsive:
crictl pods
crictl ps -a
crictl logs $(crictl ps -a --name kube-apiserver -q)
journalctl -u kubelet --no-pager -n 50 | grep -i error
sudo cat /etc/kubernetes/manifests/kube-apiserver.yaml | head -50
```

Step 3 — Fix static pod manifest

```
# Wrong flag/image in manifest causes API server to not start
sudo vi /etc/kubernetes/manifests/kube-apiserver.yaml
# Kubelet detects change within 20 seconds
crictl ps | grep apiserver
kubectl get nodes
# Should work once apiserver recovers
```

Step 4 — Diagnose controller manager and scheduler

```
kubectl get pods -n kube-system | grep -E 'controller|scheduler'
kubectl logs -n kube-system kube-controller-manager-controlplane | tail -20
kubectl logs -n kube-system kube-scheduler-controlplane | tail -20
```

Note: Static pod manifests: changes cause automatic pod restart by kubelet -- no restart needed. Use crictl when API server is down: crictl pods, crictl logs, crictl ps -a.

Test it: Broken API server manifest is fixed; kubectl get nodes works again. journalctl shows the specific error. All kube-system pods are Running.

Lab 27 — Troubleshoot Nodes: NotReady, kubelet Failure, cgroup Driver Mismatch, Drain

Domain: Troubleshooting · Topic: Node troubleshooting · Duration: 60 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubeadm

Goal

Diagnose a node transition to NotReady by identifying common causes: kubelet service failure, cgroup driver mismatch between kubelet and containerd, and containerd failure. Practice safely draining a node for maintenance.

What you'll build

Stop kubelet, observe NotReady, restart. Introduce cgroup mismatch, diagnose with journalctl, fix config.toml, drain and uncordon node.

Step 1 — Check node status baseline

```
kubectl get nodes -o wide
kubectl describe node node01 | grep -A10 Conditions
```

Step 2 — Scenario: kubelet service stops

```
# On node01:
```

```
sudo systemctl stop kubelet
# On control plane (wait 40s):
kubectl get nodes
# node01: NotReady
# Fix (on node01):
sudo systemctl start kubelet && kubectl get nodes
```

Step 3 — Scenario: cgroup driver mismatch

```
journalctl -u kubelet --no-pager -n 30
# Look for: 'failed to create containerd task' or 'cgroupDriver'
grep -i cgroup /var/lib/kubelet/config.yaml
grep -i SystemdCgroup /etc/containerd/config.toml
# Fix: ensure both use cgroupDriver: systemd / SystemdCgroup = true
sudo systemctl restart kubelet containerd
```

Step 4 — Drain node for maintenance

```
kubectl drain node01 --ignore-daemonsets --delete-emptydir-data
kubectl get nodes
# node01: Ready,SchedulingDisabled
# After maintenance:
kubectl uncordon node01
```

Note: After 40s without kubelet heartbeat, node controller marks NotReady. After 5 minutes, pods are evicted to healthy nodes. Drain before any maintenance.

Test it: Stopped kubelet causes NotReady; starting it restores Ready within 1 minute. Drain evicts pods and cordons node; uncordon re-enables scheduling.

Lab 28 — Application Logs: kubectl logs, --previous, Multi-Container, Raw Log Files on Disk

Domain: Troubleshooting · Topic: Application logs · Duration: 30 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Read logs from running containers, crashed containers (--previous), specific containers in multi-container pods, and raw log files on disk at /var/log/pods/. Log retrieval is the first step in every CKA application troubleshooting question.

What you'll build

Run noisy pod, use kubectl logs with --tail --timestamps --follow. Create crashloop pod and read with --previous. Locate raw log file on disk.

Step 1 — Basic log commands

```
kubectl run noisy --image=busybox --restart=Never -- \
  sh -c 'i=0; while true; do echo "line $i"; i=$((i+1)); sleep 1; done'
kubectl logs noisy --tail=5
kubectl logs noisy --tail=10 --timestamps=true
```

Step 2 — Crashed container logs with --previous

```
kubectl run crasher --image=busybox --restart=Always -- \
  sh -c 'echo starting; sleep 2; echo crashing; exit 1'
kubectl get pod crasher -w
# Watch CrashLoopBackOff
kubectl logs crasher --previous
```

Step 3 — Multi-container pod logs

```
kubectl logs multi-pod -c c1
kubectl logs multi-pod -c c2
kubectl logs multi-pod --all-containers
```

Step 4 — Raw log files on disk

```
ls /var/log/pods/
find /var/log/pods/ -name '*.log' | head -5
tail -20 /var/log/pods/default_noisy_*/noisy/0.log
```

Note: --previous reads the last terminated container instance -- essential for CrashLoopBackOff. Raw log files at /var/log/pods/ persist even if the pod is deleted.

Test it: kubectl logs noisy shows incrementing lines. kubectl logs crasher --previous shows crash message. Raw log file found under /var/log/pods/default_noisy_*/.

DAY 4 (½ day) — DOMAIN 5: Monitoring & Troubleshooting (Labs 29–31)

Lab 29 — Monitor Cluster and Application Usage: kubectl top, Events, Metrics Pipeline

Domain: Troubleshooting · Topic: Monitoring and observability · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Use kubectl top to check CPU and memory usage for nodes and pods, inspect cluster events to identify recent warnings, and understand how metrics flow from cAdvisor through metrics-server to kubectl top and HPA.

What you'll build

Install metrics-server, run kubectl top node and kubectl top pod (sorted by CPU/memory), inspect kubectl get events, use kubectl describe to surface warning events.

Step 1 — Install metrics-server

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml
kubectl -n kube-system patch deploy metrics-server --type=json \

-p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"-
-kubelet-insecure-tls"}]'
until kubectl top node 2>/dev/null; do echo 'waiting...'; sleep 5; done
```

Step 2 — Resource usage

```
kubectl top node
kubectl top pod -A
kubectl top pod -A --sort-by=cpu | head -10
kubectl top pod -A --sort-by=memory | head -10
```

Step 3 — Cluster events

```
kubectl get events -A --sort-by='.lastTimestamp'
kubectl get events -A --field-selector type=Warning
kubectl describe node worker01 | grep -A20 Events
```

Step 4 — Metrics pipeline raw API

```
kubectl get --raw '/apis/metrics.k8s.io/v1beta1/nodes' | python3 -m
json.tool | head -30
```

Note: Metrics flow: cAdvisor (per node) -> kubelet Summary API -> metrics-server -> kubectl top + HPA. kubectl top requires metrics-server. Events are the first place to look when a pod fails to start.

Test it: kubectl top node shows CPU and memory for all nodes. kubectl get events shows Warning events. Metrics API returns JSON with per-node data.

Lab 30 — Troubleshoot Networking: DNS, Service Selector, Endpoints, CNI, NetworkPolicy

Domain: Troubleshooting · Topic: Network troubleshooting · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Follow a structured 7-step triage chain for Kubernetes networking failures: DNS -> Service selector -> Endpoints -> CNI -> NetworkPolicy -> kube-proxy -> routing. CKA 2026 tests this methodical approach in multi-step troubleshooting questions.

What you'll build

Deploy baseline app, break DNS by patching CoreDNS, break service selector, block traffic with NetworkPolicy, fix each layer and verify restoration.

Step 1 — Deploy baseline

```
kubectl create deployment app --image=nginx --replicas=2
kubectl expose deploy app --port=80 --name=app-svc
kubectl run dbg --image=nicolaka/netshoot --command -- sleep 3600
kubectl exec dbg -- curl -s http://app-svc
```

Step 2 — Layer 1: DNS check

```
kubectl exec dbg -- nslookup app-svc.default.svc.cluster.local
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl -n kube-system logs -l k8s-app=kube-dns | tail -20
```

Step 3 — Layer 2: Service and Endpoints

```
kubectl get svc app-svc -o yaml | grep selector -A 5
kubectl get endpoints app-svc
kubectl get pods --show-labels
# Empty endpoints = selector mismatch -- compare and fix
```

Step 4 — Layers 3-7: CNI, NetworkPolicy, kube-proxy

```
kubectl get pods -o wide
# Check pod IPs assigned (CNI OK)
kubectl get networkpolicy -A
# Check for blocking policies
kubectl -n kube-system get pods -l k8s-app=kube-proxy
```

Note: Always start with DNS -- 80% of Kubernetes networking problems are DNS. If DNS works but curl fails: check Service selector vs pod labels. If endpoints exist but curl fails: check NetworkPolicy or kube-proxy.

Test it: Baseline curl to app-svc works. Selector mismatch: endpoints empty, curl fails. After fixing selector: endpoints repopulated, curl works again.

Lab 31 — Troubleshoot RBAC (403 Forbidden) and Scheduling Failures (Pending Pod)

Domain: Troubleshooting · Topic: RBAC and scheduling troubleshooting · Duration: 45 min · Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

Goal

Diagnose and fix two of the most common CKA exam question types: a 403 Forbidden error caused by a missing ClusterRoleBinding, and a pod stuck in Pending due to a scheduling constraint (taint, node selector, or resource exhaustion).

What you'll build

Create SA + pod that fails with Forbidden, diagnose with auth can-i, add missing binding. Create pod that is Pending, diagnose with kubectl describe, fix constraint.

Part A — RBAC: diagnose 403 Forbidden

```
kubectl create serviceaccount dev-sa
# Pod using dev-sa attempts to list pods:
kubectl exec sa-pod -- kubectl get pods
# Error from server (Forbidden)
kubectl auth can-i list pods --as=system:serviceaccount:default:dev-sa
# no
```

Part A — Fix: create RoleBinding

```
kubectl create clusterrolebinding dev-sa-view \
  --clusterrole=view \
  --serviceaccount=default:dev-sa
kubectl auth can-i list pods --as=system:serviceaccount:default:dev-sa
# yes
kubectl exec sa-pod -- kubectl get pods
```

Part B — Scheduling: diagnose Pending pod

```
kubectl run pending-pod --image=nginx
kubectl get pod pending-pod
# Pending
kubectl describe pod pending-pod | grep -A20 Events
# Look for: Insufficient cpu, node(s) had taint, MatchNodeSelector
```

Part B — Fix scheduling constraints

```
# Taint causing failure:
kubectl describe node worker01 | grep Taints
kubectl taint node worker01 <key>-
# Remove taint
# Or add toleration to pod:
tolerations:
- key: <key>
  operator: Exists
  effect: NoSchedule
kubectl get pod pending-pod -o wide
# Now Running
```

Note: kubectl describe pod is ALWAYS the first command for a Pending pod -- Events shows exactly why. kubectl auth can-i --as is the fastest way to test what a ServiceAccount can do.

Test it: After adding ClusterRoleBinding: SA can list pods, no Forbidden error. After fixing taint/selector: pod transitions from Pending to Running.

Troubleshooting Cheat-Sheet

Symptom	Likely cause & fix
Node is `NotReady`	Check `kubectl describe node` and `journalctl -u kubelet`. Usually a kubelet crash, containerd stop, or bad manifest. `systemctl start kubelet` often resolves it.
`kubectl` commands all fail (connection refused)	API server is down. SSH to control-plane node: `crictl ps | grep kube-apiserver`. Check the static pod manifest in `/etc/kubernetes/manifests/`.
Pod stuck in `Pending`	No schedulable node — check taints, resource limits, node availability: `kubectl describe pod <name>` → Events section.
Pod in `CrashLoopBackOff`	Container exits non-zero repeatedly. Read `kubectl logs <pod> --previous` to see the last failure output.
`ImagePullBackOff`	Wrong image name, tag, or missing `imagePullSecret`. Check `kubectl describe pod` Events and fix the image field.
`OOMKilled`	Container exceeded its memory limit. Raise `resources.limits.memory` or reduce the workload's memory usage.
etcd backup fails	Ensure `ETCDCTL_API=3` is set. Check `--endpoints`, `--cacert`, `--cert` and `--key` flags. Verify with `etcdctl endpoint health`.
Service not routing traffic / empty endpoints	Selector/label mismatch. Check `kubectl get endpoints <svc>` and compare selector to pod labels: `kubectl get pod --show-labels`.

CoreDNS not resolving names	Check CoreDNS pods are Running: `kubectl get pods -n kube-system`. Inspect the Corefile: `kubectl get cm coredns -n kube-system -o yaml`.
PVC stuck `Pending`	No matching PV or no default StorageClass. Check `storageClassName`, `accessModes` and requested capacity.
Ingress returns 404 / 503	No matching host/path rule, or backend Service is down. `kubectl describe ingress <name>` and check `kubectl get endpoints` for the backend.
`Error from server (Forbidden)` / `403`	RBAC denial. Check ServiceAccount, Role and RoleBinding: `kubectl auth can-i <verb> <resource> --as=<identity>`.
Static pod not starting after manifest edit	Kubelet watches `/etc/kubernetes/manifests/` and auto-restarts static pods within ~30 s. If still down, check for YAML syntax errors.
Cluster upgrade breaks: worker node stuck	Run `kubeadm upgrade node` on the worker, then `apt-mark unhold` and reinstall `kubelet kubectl` before `systemctl restart kubelet`.

Glossary

Term	Meaning
Control Plane	The set of components — kube-apiserver, etcd, kube-scheduler, kube-controller-manager — that store and reconcile cluster state.
Worker Node	A machine (VM or bare-metal) that runs Pods; managed by kubelet and kube-proxy.
etcd	The distributed key-value store that holds all Kubernetes cluster state. Back it up with `etcdctl snapshot save`.
kubelet	The node agent — watches the API server for Pods assigned to this node and starts/stops containers.
kube-proxy	Programs iptables or IPVS rules on each node to implement Service ClusterIP and NodePort routing.
kubeadm	The cluster bootstrapper: `kubeadm init` on the control plane, `kubeadm join` on workers, `kubeadm upgrade` for version bumps.
CNI (Container Network Interface)	The plugin standard for assigning Pod IPs. Calico (BGP/VXLAN) and Cilium (eBPF) are the most common CKA choices.
CRI (Container Runtime Interface)	The gRPC interface between kubelet and the container runtime (containerd or CRI-O). Debug with `crictl`.
CSI (Container Storage Interface)	The plugin standard for storage drivers. StorageClass references a CSI provisioner for dynamic PV creation.
CRD (CustomResourceDefinition)	Registers a new object kind with the API server, extending Kubernetes without

	modifying core code.
Static Pod	A Pod whose manifest lives in `/etc/kubernetes/manifests/`; managed directly by kubelet — control-plane components run this way.
Deployment	Manages a desired number of Pod replicas; handles rolling updates, rollbacks and self-healing.
ReplicaSet	The low-level object a Deployment creates to maintain the correct Pod count.
DaemonSet	Schedules exactly one Pod per matching node — used for CNI agents, log collectors, monitoring exporters.
StatefulSet	Like Deployment but with stable Pod names (db-0, db-1), ordered start/stop and per-replica persistent storage.
HPA (Horizontal Pod Autoscaler)	Adjusts replica count automatically based on CPU/memory metrics from metrics-server.
Service (ClusterIP)	A stable virtual IP + DNS name in front of matching Pods; reachable inside the cluster only.
Service (NodePort)	Opens a port (30000–32767) on every node to expose Pods outside the cluster.
Ingress	Layer-7 HTTP/HTTPS routing rules (host/path → Service); requires an Ingress controller like ingress-nginx.
Gateway API	The modern successor to Ingress — role-oriented (GatewayClass, Gateway, HTTPRoute); more expressive and extensible.
NetworkPolicy	Allow-list firewall for Pod-to-Pod and Pod-to-external traffic; default is fully open.
CoreDNS	The in-cluster DNS server; resolves `<service>.<namespace>.svc.cluster.local` and forwards external queries upstream.
ConfigMap	Non-secret key/value data injected into Pods via env vars or volume mounts.
Secret	Like ConfigMap but base64-encoded; access controlled by RBAC. Not encrypted at rest by default — enable EncryptionConfig to harden.
PersistentVolume (PV)	A piece of cluster-level storage resource provisioned by an admin or dynamically by a StorageClass.
PersistentVolumeClaim (PVC)	A namespaced request for storage; binds to a matching PV.
StorageClass	Defines a CSI provisioner and parameters for dynamic PV creation on demand.
Helm	The Kubernetes package manager — installs and upgrades applications from versioned charts.
Kustomize	Layer YAML overlays on a base without templating; built into `kubectl apply -k`.
Role / ClusterRole	List of allowed verbs (get, list, create…) on resources; Role is namespace-scoped, ClusterRole is cluster-wide.
RoleBinding / ClusterRoleBinding	Attaches a Role or ClusterRole to a subject

	(ServiceAccount, user or group).
Taint / Toleration	Taints on nodes repel Pods that don't have a matching toleration in their spec.
Node Affinity	`nodeSelector`'s successor — allows required (hard) and preferred (soft) scheduling constraints using label selectors.
crictl	The low-level CRI client for inspecting containers and images when `kubectl` is unavailable (API server down).
etcdctl	The CLI for backing up (`snapshot save`) and restoring (`snapshot restore`) the etcd data store.
kubectl --dry-run=client -o yaml	Generates a manifest without creating the resource — the `$do` alias used throughout these labs.
killer.sh	The official CKA exam simulator — realistic cluster tasks in a timed, browser-based environment.